LA RECURSIVITE

A quoi sert la décomposition des programmes en modules ?

- Lisibilité du programme.
- Lorsqu'un algorithme doit effectuer plusieurs fois la même tâche, il sera mieux d'isoler cette tâche dans un module et de l'appeler aux moments opportuns
- ….

Comment un module est utilisé dans un programme ?

➔ *Il est appelé par le programme principal ou par un autre module*

Que ce passe-t-il si un module serait appelé dans son propre corps ?

Exemple : *expliquer un mot en utilisant un dictionnaire*

1- Chercher la définition du mot dans le dictionnaire
2- Expliquer les mots non clairs en utilisant le même dictionnaire

Définition :

Un sous-programme est dit **récursif**, s'il fait appel à lui-même.

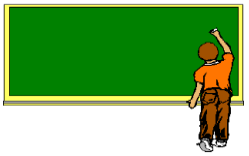
Cet appel doit respecter les conditions suivantes:

- Les valeurs utilisées en paramètre convergent, d'un appel à un autre, vers **un cas particulier**.
- Le sous-programme traite un cas particulier et un cas général et doit s'arrêter si le cas particulier est atteint.

Activité :
Ecrire une fonction qui permet de calculer le factoriel d'un nombre n donné.

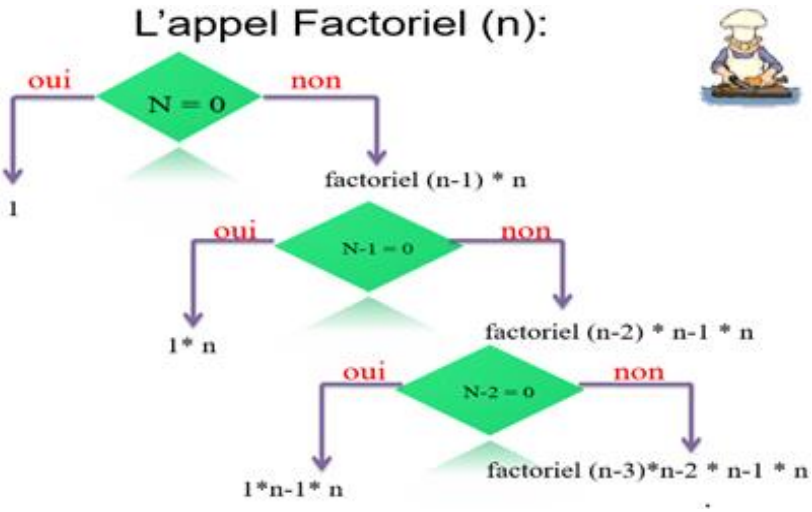
Factoriel (0) = 1
Factoriel (1) = 1
Factoriel (2) = 1 * 2
Factoriel (3) = 1 * 2 * 3
Factoriel (4) = 1 * 2 * 3 * 4
Factoriel (5) = 1 * 2 * 3 * 4 * 5
Factoriel (6) = 1 * 2 * 3 * 4 * 5 * 6

n! = 1*2*3*....*n



En générale : n!= (n-1)! * n
Et 0!= 1

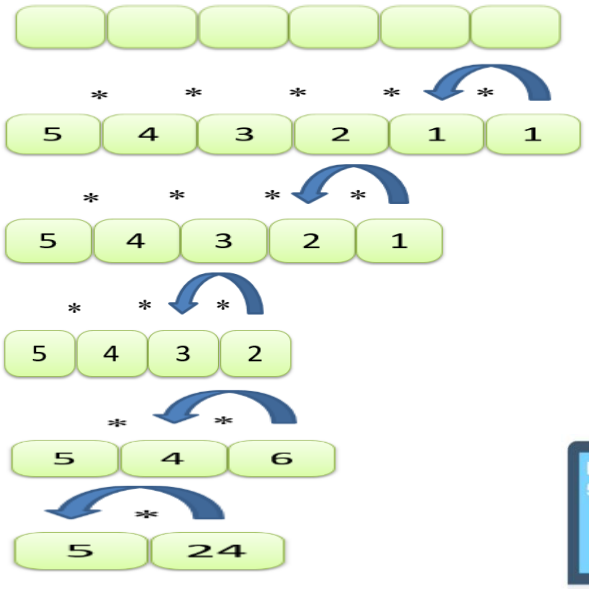
Utilisons cette règle pour définir la fonction **Factorielle (n)**: ***n! = (n-1)! * n***
0!= 1
Le cas n=0 est un cas particulier, il doit donc être traité à part.
Donc le programme doit contenir l'instruction:
Si (n=0) OU (n=1) Alors retourner 1
Pour les autres valeurs de n, ils sont calculés de la même manière suivante: ***n! = (n-1) = (n-2)!* n-1 * n***
= (n-3)!* (n-2)* (n-1) *n
. . .
= 0!* 1*2*3* ... *(n-1)* n
= 1*2*3* . . . *n



Algorithme de la fonction Factoriel :

```
Fonction factorielle (n: entier): entier
Début
Si (n=0) Alors
      Retourner 1
   Sinon
     Retourner n* Factorielle(n-1)
   Fin Si
Fin
```

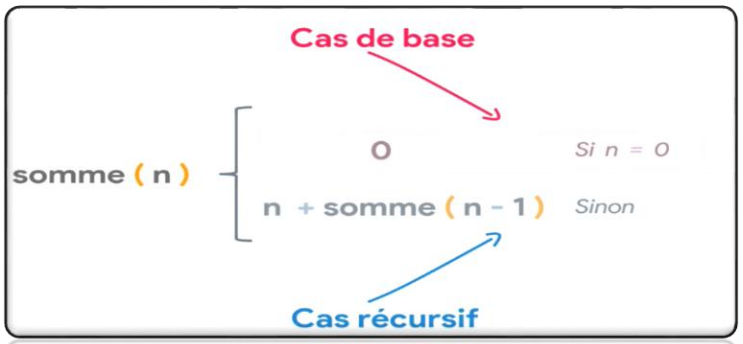
Remplir la pile :



Activité 2 :

Ecrire un algorithme récursif qui calcule la somme des entiers de 1 à n.

S= 1 + 2 + 3 +….. + n




```
Fonction Somme (n : entier) : entier
Début
Si n= 0 alors
    Retourner 0
Sinon
    Retourner n + Somme(n-1)
Fin si
Fin
```

<u>Activité 3 :</u>

Ecrire un algorithme d'un module récursif qui calcule tous les termes de la suite Fibonacci, inférieurs ou égaux à n. (n : entier positif donné).

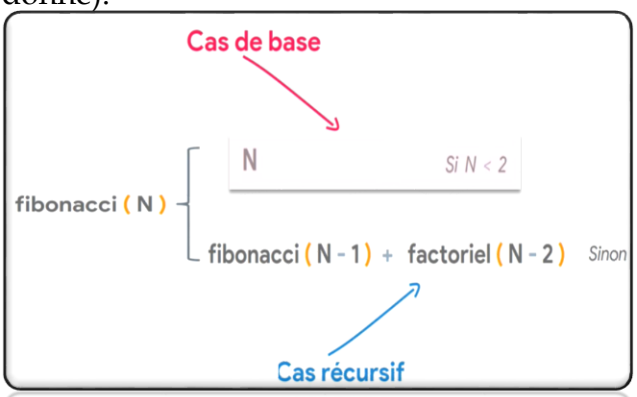
La suite de Fibonacci est définie comme :

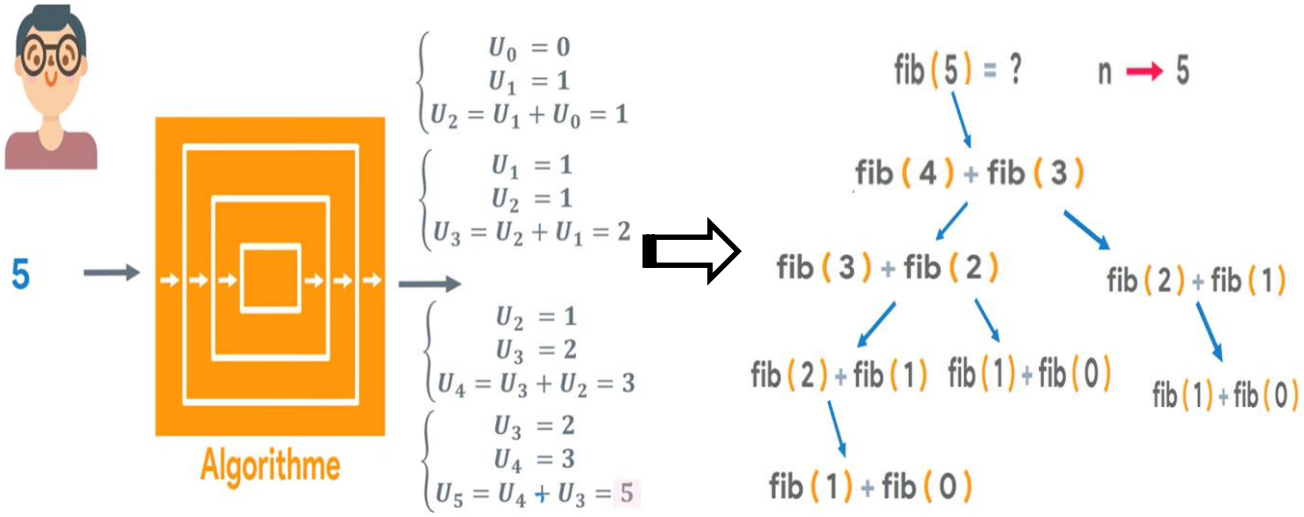
$$\begin{cases} U_0 = 0 \\ U_1 = 1 \\ U_{n+2} = U_{n+1} + U_n \end{cases}$$

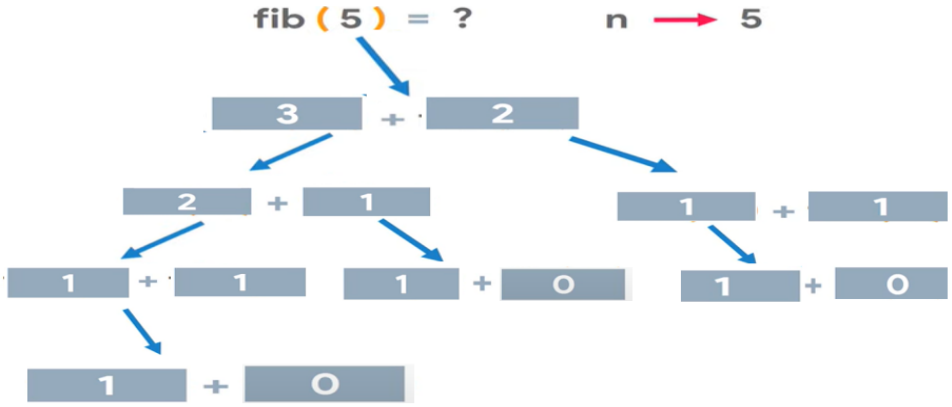
Exemple: Entrer un nombre: **5**

la suite fibonacci est :

0 , 1, 1, 2, 3, 5







Algorithme de la fonction Fib:

```
Fonction fib (n: entier): entier
Début
 Si n <= 1 alors
      Retourner n
 Si non
      Retourner (fib (n-1) + fib (n-2))
Fin Si
Fin
```